

- 알고리즘 문제해결 전략
1. Linked List ^{과목과 과목 연결}
- 노드 : 실제 정보를 담고 있는 데이터 / 링크 : 노드간의 연결을 나타내는 포인터
 - 동적인 자료구조(필요할 때 할당, 미리 크기 지정할 필요X) / 순차적 접근 가능한 자료구조(임의 접근 불가능) / 불연속 메모리 사용(물리적 순서 없고 불연속적으로 저장, 링크에 의해 순서 유지)
 - 단일 연결 리스트(단방향 : data, next pointer) / 이중 연결 리스트(양방향 : data, next pointer, prev pointer)
2. Stack ^{용량 증가}
- LIFO(Last In Last Out) / 입구와 출구가 하나인 자료구조
 - push : 데이터 추가 / pop : 가장 최근에 추가한 데이터 뺌
 - 배열로 구현 : 스택 크기가 미리 정해져 있음, 들어가는 데이터 개수의 변동 폭이 적을 때(즉, 크기 가늠 가능할 때), 적절할당
 - 리스트로 구현 : 연결리스트의 특성 상 스택 크기가 자유로움, 들어가는 데이터 개수 가늠 어려울 때 (성능 면에서 배열 보다 우수- 훨씬 간단함), 동적할당, 단일 연결리스트로 구현

3. Queue ^{용량 증가}
- FIFO(First In First Out) / 입구와 출구가 따로 있는 자료구조 / front(출구) rear(입구)
 - put : 데이터 추가 / get : 가장 나중에 추가된 데이터 뺌
 - 배열 / 리스트로 구현 시 스택과 동일 / 차이점 : 이중 연결리스트로 구현

4. Tree ^{사이클이 없다}
- 노드(vertex/점)와 링크(edge/간선)로 구성된 비선형 자료구조 / ex. 윈도우 디렉터리구조(가장 전형적인 트리구조)
 - 특징 : root (최 상위 node) 제외한 모든 node는 하나의 부모를 가짐 / node는 여러 개의 자식 가질 수 있음 / 한 node로부터 다른 node로 가는 경로(Path)는 유일함
 - Leaf node : 자식 없는 최하위 노드(=terminal, external) / Internal node : leaf node 아닌 노드(non-terminal)
 - subtree : 트리 부분집합 / Level : root에서 특정노드로 가는 경로의 노드 수 / Height : 가장 높은 레벨

5. Binary Tree ^(노드 = 2 이하)
- 모든 Internal node가 두 개 이하의 자식을 갖는 트리(최대 두 개 : left child, right child)
 - 가장 쓰임새가 많은 트리 (Parse Tree(수식계산) / Heap(정렬에 사용) / Binary Search Tree(검색에 사용))

Full Binary Tree : 마지막 레벨을 제외한 모든 레벨에 노드가 꽉 차있음(자식이 모두 둘)

$2^{d-1} \leq N \leq 2^d - 1$ 이면(log2 취해서 증명) $d = \lceil \log_2 N \rceil + 1$ (레벨: d, 노드: n개)

Complete Binary Tree : 모든 레벨에 노드가 꽉 차있음

6. Tree Traversal

- 트리가 비선형구조이기 때문에 전체 노드를 방문하는 방법이 필요(트리의 모든 노드 방문) - 방법에 차이가 있음
- stack 기반 : pre-order / post-order / in-order queue 기반 : level-order
- pre-order traversal : root -> left -> right 123
- in-order traversal : left -> root -> right 213
- post-order traversal : left -> right -> root 312
- Parse Tree: 수식을 연산순서에 따라 트리로 구성. root(operator: 모두 non-terminal) / child(operand: 모두 leaf)

7. 정렬 알고리즘

- 데이터 집합을 정해진 순서대로 재배열 하는 것. 오름차순(ascending order) / 내림차순(descending order)
- 키 값(비교 대상, 정렬 기준) / 레코드(교환 대상, 하나의 정보 단위(여러 개 필드가 합쳐진 것))
- 장소에 따라 : 내부정렬(컴퓨터 메모리 내에서 작동): 정렬할 개수가 적어서 주기억장치에 모두 적재시켜 놓고 주기억장치 내에서 정렬하는 방법 / 특정 인덱스로 바로 읽어오기 가능
- 외부정렬(테이프(순차적 접근), 하드디스크): 자료개수를 주기억장치가 수용하지 못해서 보조기억장치에 저장해두고 일부만 차례로 적재해서 사용하는 방법

* 메모리 영역, ← 가상메모리 영역 → 프로그램 동작시 메모리에 프로그램이 동작하기 위한 가상메모리 공간 생성
 목적에 따라 이 영역은 상위 / 하위 메모리를 나뉜다

① Stack : 임시변수를 저장하는 특정영역 / 순서있다 (LIFO)

(하위영역) 함수에서 새로운 변수 선언시 stack이 저장, 함수의 종료시 stack이 null로 자동으로 메모리 해제된다.

② Heap : stack과 달리 CPU에서 자동으로 관리해주지 않는 영역 / 프로그래머가 직접 영역 할당

(하위영역) 메모리 주소값만으로 참조 / malloc(), calloc() 등으로 메모리 할당 하고 직접 해제 → 안하면 메모리 누수 발생 (memory leak)

장점: 상황에 따라 원하는 크기만큼 메모리 할당 가능. & 메모리 크기 조절 가능.

CF. 프로그램에서
 힙 영역과 스택 영역
 해당 영역에서
 종료되기 전까지
 그 데이터를 읽고
 이걸 프로그램에
 명시적 해제.
 해당 프로그램 종료
 운영체제
 메모리 리소스 반환
 통한 메모리 해제

CF.

메모리에 고려해야 할 문제.

① 속도제한 문제 → 캐시 메모리 등장.

② 용량제한 (메시나온기 회차링 기법)

강제된 속도제한 문제.
 ex. CPU < 메모리 < 하드디스크
 1 150
 (하드디스크와 관련된
 메모리는 4GB가 넘지 않음)

* 캐시 메모리 (cache memory) ← SRAM으로 많음

= 메인 메모리에 있는 명령어와 데이터 중 일부를 가져와 임시적으로 저장하여 CPU에서 필요할 때 해당 정보의 신속한 제공이 가능도록 만들어진 장치.

- CPU와 메인메모리 사이에 데이터 처리 속도 차이가 크기 때문에 CPU 활용 제어로 온다. ∴ CPU 캐시와 메인 메모리

캐시와 메인 메모리
 데이터 속도 차이 극복
 데이터 전송 속도 차이 극복
 데이터 전송 속도 차이 극복

- 교체 알고리즘 → 작동: CPU가 현재 필요한 정보를 캐시 메모리에서 찾을 때 있을 경우

→ 상태: 없음 / 있음

캐시 메모리에서 공간 마련을 위해 데이터 제거, 어떤 데이터 제거할지 결정하는 것.

→ LRU, LFU, FIFO 등 존재. → 데이터 교체 알고리즘 (캐시에 정보 없을 때 캐시에서 제거 방법)

Least

- 교환 방식에 따라 : 직접정렬(레코드 전체 이동하여 정렬) / 간접정렬(별도로 존재하는 키 값의 인덱스만을 정렬)
- Stability : 같은 키 값을 가지는 여러 레코드가 정렬 후에도 그 상대적 순서를 유지하는지 여부

1. Selection Sort

- 최소값 찾아 앞으로 이동시킴 / 정렬되지 않은 부분에서 최소값을 찾아 정렬된 부분 뒤에 더함(앞부분부터 완전정렬)
- complexity : $O(n^2)$ / Worst case = Best case = $O(n^2)$ ($n*(n-1)/2$)
- 입력 자료에 둔감 : 교환 횟수가 적으므로(메모리 복사가 적으므로) 큰 레코드에 적합

2. Insertion Sort

- 현재 레코드를 일부 정렬된 부분의 적절한 위치로 옮김 / 현재 수와 바로 앞의 수 비교(앞부분부터 상대적 정렬)
- complexity : $O(n^2)$ / Worst case = $O(n^2)$ (reverse) / Best case = $O(n)$ ((n-1)->sorted)
- 입력 자료에 민감 : 거의 정렬된 문제에서는 빠름

3. Bubble Sort

- 인접 요소의 비교/교환을 통해 최대값을 제일 뒤로 보냄(뒷부분부터 완전정렬 되는 모양)
- complexity : $O(n^2)$ / Worst case = Best case = $O(n^2)$
- 비효율적 : 교환이 3번의 operation으로 구성(swap) / 잘 사용하지X / 성능 좋지않음 / 느림 / 시간 오래걸림

4. Quick Sort

- 분할(partition) 알고리즘: 축값(pivot value)을 중심으로 왼쪽은 축값보다 작은 값, 오른쪽은 축값보다 큰 값으로 배열
- 크기가 1보다 클 동안(1 될 때까지) partition을 재귀적으로 반복(재귀적으로 분할되어 가는 과정) / 멀리 떨어져 있는 요소를 교환하기 때문에 효율적임
- Worst case = $O(n^2)$ (분할이 N-1 : 1로 이루어질 때 (순차배열, 역순배열)=Degenerated Case)
=> 문제 : (Recursive Quick Sort일 때) Stack Overflow Error 발생
해결법 : Non-recursive로 사용하여 stack overflow 방지 / 개선방법 3가지 사용
개선법 1. 난수분할: pivot value를 임의로(random하게) 정함 <<1,2,3 모두 순차/역순(최악)도 $O(N \cdot \log N)$ 나옴>>
2. Median-of-Three Partition: 처음, 가운데, 끝 값 중 중위값을 찾아 그것을 pivot value로 하는 방법
3. Subfiles(2번 + Insertion Sort): 퀵정렬은 큰 데이터 집합에 유리한데 이때 작은 데이터 집합에는 (↑최적방법) 단순한 삽입정렬이 유리하므로 파티션을 해가며 작은 구간이 되면 삽입정렬로 대체하여 재귀의 깊이 줄임(스택사용 줄임=메모리 사용 줄임)
/ Best case = $O(N \cdot \log N)$ (분할이 정확히 절반씩 나누어질 때) ($T(N) = 2 * T(N/2) + N$)
- 결국 분할을 절반에 가깝게 하는 것이 성능의 관건

5. Heap Sort

- 힙: 키 값의 크기가 우선순위가 됨 / Binary Tree로 구성 / 부모노드 > 자식노드 / root에 가장 큰값
- 힙을 생성하여 힙에 자료가 하나도 남지 않을 때까지 extract함(역순저장 = 뒷쪽부터 최대값 저장)
- complexity : $O(N \cdot \log N)$ => 삽입, 선택정렬 보단 성능 좋음 / 퀵정렬에 비해서는 느림 / 입력자료에 둔감
- 힙정렬은 부가메모리를 전혀 사용하지 않음(힙부분과 입력배열, 결과배열의 경계구분이 명확하므로) space complexity = 0
- 하향식(top-down)힙 생성(N개 자료insert) < 상향식(bottom-up)힙 생성(N/2개 자료downheap) 더 효율적(빠름)

6. Merge Sort

- 두 개 이상의 정렬된 자료집합을 하나의 정렬된 자료집합으로 합치는 것
- 대상 자료집합이 정렬된 상태이므로 순차적으로 비교하며 작은자료를 뽑음(병합 대상이 되는 자료와 순차적 비교)
- complexity(time) : $O(N \cdot \log N)$ => 크기M과 크기N의 단위를 병합하는데 M+N비교 필요 $T(N)=2T(N/2)+N$ (퀵정렬)
- complexity(space) : $O(N)$ => 병합 결과를 저장하기 위한 N개의 공간이 필요(비효율적) / 연결리스트는 $O(1)$ 이유는 이전노드만 참조하면 됨
- 입력 자료에 둔감(역순/순차 정렬 등과 무관)
- 순차적 접근만 사용하기 때문에 (즉, Random Access X) 순차적 접근만 가능한 장치(tape같은)의 외부정렬에 사용
- Random Access의 성능이 떨어지는 경우 효과적(Sequential Access 이므로)
- Stability 있음 (값이 같더라도 정렬 후 상대적 위치를 유지하는 것)

7. Shell Sort

- 삽입정렬의 문제점(비교/교환 횟수 많음 $O(N^2)$)을 개선한 것
/ 어떤 수열을 선택할 것인가가 성능의 관건 ($h(N) = 3h \cdot (N-1) + 1$)
- complexity : $O(N(\log N)^2)$ or $O(N^{1.25})$ ($O(N \cdot \log N)$ 보다는 크나 단순한 알고리즘이므로 속도 빠름, 입력자료 둔감)
- space complexity = 0 => 부가 메모리를 사용하지 않음(입력배열 그 자체 제외하고 다른 부가메모리 사용X)

8. Distribution Counting

- 같은 키가 많이 중복되는 정수 자료집합에 적용 가능한 매우 단순한 알고리즘
- 특정 키가 출현하는 빈도와 누적분포 이용
- complexity(time) : $O(N)$ => 단일루프로만 이루어짐(성능 우수하나 메모리 공간 사용 많이함)
- complexity(space) : $N+M$ => 결과를 저장할 배열 크기 N + 키 값의 분포를 저장할 count배열 크기 M
- 중복된 키가 많고 키의 범위가 비교적 작은 경우 유리(이때 메모리 낭비X. but 크면 낭비多)
- 양의 정수 자료만 정렬 가능 => 정수 값이 그대로 인덱스 요소가 되기 때문

9. Radix Exchange Sort

- Quick Sort와 유사한 partition의 재귀적 반복 / partition의 기준은 비트(0 or 1) / 양의 정수만 정렬 가능
/ MSB(Most Significant Bit: 가장 자릿수 큰값) -> LSB(가장 자릿수 작은값) ex) 010101을 000111로 만드는 것
- 최상위 비트부터 bit partition 실행해서 분할된 영역에 대해 재귀호출
- 퀵정렬과 유사 / 차이점 : 분할의 축값(pivot value)이 없음
- complexity(time) : $O(N \cdot \log N)$ / complexity(space) (스택 사용량) : $O(W)$ $3W+3$ (W =자료의 비트 수)

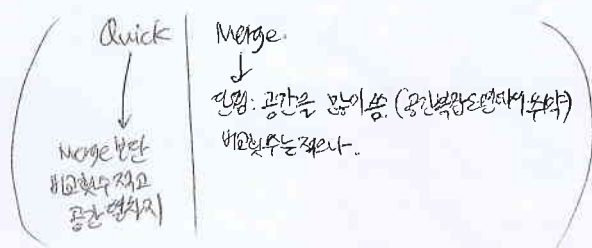
10. Straight Radix Sort

- 기수교환정렬과 반대로 최하위자리(LSB)부터 최상위자리(MSB)로 올라가면서 Stability 있게 정렬 (LSB->MSB)
- 기수교환정렬과 마찬가지로 양의 정수에서만 가능
- 메모리카운트(비트 수)를 얼마씩 잡느냐에 따라 분포수세기 실시횟수 달라짐(메모리 늘리느냐 or 속도 빠르게하냐)
- complexity(time) : $O(N)$ => $(W/M) \cdot \text{분포수세기 시간}$ -> 선형 정렬 알고리즘(Linear Sorting Algorithm)
- complexity(space) : $N + 2^M$ => 분포 결과 저장 배열 크기 N + 분포 저장할 count배열 크기 2^M
- M 크게 하면(분포수 세기 횟수 적어지면) 성능 좋아짐. but 메모리 차지 많아짐

* Radix(기수)를 기반으로 하는 정렬은 입력배열에 둔감함
random(난수)배열, sorted배열, reverse배열 차이 없음 => 입력배열에 상관없이 비트만 보고 따지기 때문

[정렬 알고리즘 정리]

Algorithm	time complexity	space complexity	worst case	best case	stability
Selection Sort	$O(N^2)$	0	$O(N^2)$	$O(N^2)$	Y
Insertion Sort	$O(N^2)$	0	$O(N^2)$	$O(N)$	Y
Bubble Sort	$O(N^2)$	0	$O(N^2)$	$O(N^2)$	Y
Shell Sort 퀵 개선	$O(N(\log N)^2)$ or $O(N^{1.25})$	0			N
Quick Sort	$O(N \cdot \log N)$	$2N+2$	$O(N^2)$	$O(N \cdot \log N)$	N
Heap Sort	$O(N \cdot \log N)$ 개선	0			N
Merge Sort 외부정렬 극히 느림	$O(N \cdot \log N)$	X			(Y)
Radix Exchange Sort 외부정렬 극히 느림	$O(N \cdot \log N)$ 개선	$3W+3$			N
Distribution Counting	$O(N)$	$N+M$			Y
Straight Radix Sort	$O(N)$ 개선	$N+2^M$			N < Y



8. 검색 알고리즘

- 검색 : 자료집합에서 원하는 데이터를 찾아내는 것 / 세가지 요소 : 검색, 삽입, 삭제
- 자료의 조직화 : 재조직 비용이 듦. 삽입/삭제 비용 $\uparrow$ but 검색 속도 $\uparrow$ $\rightarrow$ 자료를 조직화 하는 것이 좋음
- 자료집합의 조직화여부에 따라 성능이 달라짐 / 조직화 방법에 따라 알고리즘이 달라짐

1. Sequential Search worst $O(N)$

- 무순서로 저장된 자료집합에서 검색하고자 하는 키를 처음부터 차례로 비교해서 찾아내는 방법
- 배열로 구현 $\Rightarrow$ complexity : 삽입= $O(1)$ / 검색= $O(N)$ (worst) / 제거= $O(N)$
- 리스트로 구현 $\Rightarrow$ complexity : 삽입= $O(1)$ / 검색= $O(N)$ / 제거= $O(1)$ $\rightarrow$ 배열로 구현할 때 보다 효율적

2. Binary Search

- 정렬된 자료집합을 가지고 구간크기 > 0 일 때, 키 값과 중앙값을 비교 / 같: 찾음 / 키 > 중앙 : 오른쪽 / 키 < 중앙 : 왼쪽
- complexity : 삽입 = $O(N)$ (삽입정렬의 한 스텝과 동일한 방식 / worst = 최소값 삽입할 때)
- / 검색 = $O(\log N)$ (구간 이등분하여 줄여나감 / worst = 자료에 없는 값 찾을 때)
- / 제거 = $O(N)$ (삭제시간 = 검색시간($O(\log N)$) + 자료가동시간($O(N)$))

3. Interpolation Search

- 자료집합이 정렬된 선형분포를 가진다(linear하다)고 가정하고 내분법(Interpolation)에 의해 mid 값을 찾음
- 나머지 과정은 이분검색과 동일 / 산술연산 가능한 숫자 키만 가능(정수, 실수만 가능)
- complexity : 삽입 = $O(N)$ (이분검색과 동일)
- / 검색 = $O(\log(\log N))$ (linear만 성능 $\uparrow$ / 실수연산 성능 $\downarrow$ / 환경에 따라 다름)
- / 제거 = $O(N)$ (이분검색과 동일)

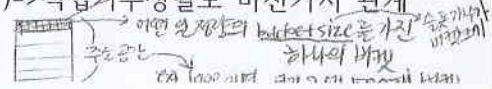
9. Binary Search Tree (BST)

- 이진트리를 이용하는 검색방법 / 부모노드는 왼쪽 subtree의 모든 노드보다 큼 / 부모노드는 오른쪽 subtree의 모든 노드보다 작음 (즉, 부모기준 좌:小 우:大)
- 여기서도 In-order / Pre-order / Post-order Traversal $\Rightarrow$ In-order Traversal했을 때 정렬된 순서
- 방법 : 검색할 키 값을 가지고 root부터 크기비교를 하면서 leaf node까지 탐색 / 같으면 찾음. 없으면 자료존재X
- 트리의 높이는 이진트리 성능에 중요한 요소 $\Rightarrow$ 좌우 균형이 맞을 때 가장 성능 좋음(+트리 높이 낮음)
- 자료의 입력순서에 의해 이진트리 모양이 결정 $\Rightarrow$ 순차/역순 일 때 트리의 불균형 심해져 순차검색과 동일한 성능
- complexity : 삽입(항상 leaf에 삽입) = 검색 = 삭제(leaf면 간단, but 내부노드면 대체노드 선택) = $O(\log N)$ (평균)
- complexity : 삽입 $\Rightarrow$ worst = $O(N)$ (한 줄로 늘어설 때) best = $O(\log N)$ $\Rightarrow$ 삽입 시 트리의 높이만큼 비교
- / 검색 $\Rightarrow$ worst = $O(N)$ (한 줄로 늘어설 때) best = $O(\log N)$ $\Rightarrow$ 검색 시 트리의 높이만큼 비교
- / 삭제 $\Rightarrow$ $O(\log N)$ $\Rightarrow$ 삭제될 노드를 대체할 후보를 찾는 데 드는 비용(오히려 불균형한 경우 찾기 쉬움)
- 결국 트리의 균형을 맞추는 것이 중요(성능이 트리의 균형에 좌우됨) $\rightarrow$ 때문에 Balanced Tree가 연구
- * 이진트리 정렬 : In-order Traversal하면 정렬된 순서가 나옴을 이용하여 정렬
- complexity : best case= $O(N \log N)$ (자료 입력 순서가 균형이 맞춰진 경우) / worst case= $O(N^2)$ (정렬/역순)

10. Hash

- hash function: 키 값을 수치(양의 정수)로 변환하는 함수 (클러스터가 적고 자료가 퍼진 것=가장 잘 만든 해쉬함수)
- hash table: 자료를 해쉬 함수의 결과 위치에 배치하는 방법(ex. 주소와 우편번호, 우체국 우편물 분류)
- best= $O(1)$ 의 성능 가지는 효율적인 검색구조(자료 개수에 상관없이 일정) / worst= $O(N)$
- $\Rightarrow$ 찾고자 하는 자료는 해쉬함수 연산으로 한번에 찾기 가능 / 해쉬함수의 선택과 해쉬테이블 크기가 중요한 요소
- collision: 서로 다른 키의 해쉬연산 결과가 같을 때 하나의 해쉬테이블에 들어가면서 충돌이 일어남 $\Rightarrow$ 선형탐사법(linear probing=open addressing) or 연결법(separate chaining)으로 해결
- 선형 탐사법: 충돌발생시 순차적으로 탐사하여 빈 곳을 찾음 / 입력자료 $\uparrow \rightarrow$ 연속된 cluster $\uparrow$, 비효율적 (hash map)
- 무조건 자료수 < 해쉬테이블 / 예상 입력 자료수의 2배 이상의 해쉬테이블 필요 / 정적인 자료 검색에 유리 (cluster=동일데이터 묶음)
- 연결법: 충돌발생시 해당 위치의 리스트에 추가 (해쉬테이블이 연결리스트의 배열로 이루어짐) (chain map)
- 자료수 > 해쉬테이블 상관X / 입력자료 수보다 적은 해쉬테이블 가능 / 동적인 상황에 적합 / 범용 사용 가능
- memory/performance trade-off: 메모리 높게 잡으면 성능 $\uparrow$ (클러스터 적어지니까) $\Rightarrow$ 직접가수정렬도 마찬가지 관계

버킷(Bucket): 해쉬함수로 계산된 값에 데이터를 저장시키기 위한 기법(동적인 해쉬함수로 계산된 값에 따라 버킷이 변함)



* 해시 함수 → 해시 함수 종류와 테이블 크기가 중요하므로 각함, 목적점: 많은 공간 활용, 양비 / 충돌

양동이 적고, 빠른 계산은 한수되어야 하며, 테이블 내 데이터(주소)가 고르게 분포해야 좋은 해시함수.
(크기 테이블 ↓) 가장 단순한 것!!!

① 제산 함수: 가장 일반적인 함수.

정수인 키를 버킷 크기로 나눈 나머지를 해시주소로 사용.

$h(Key) = Key \% M$ (이때 M 값은 '소수'인 것이 좋다. 배열 내 고른 부분을 위해)

② 표정 함수: 탐색 키 전체를 해시주소로 함.

주소를 표에서 나눠서 수를 더해서 (ex. $\square + \square + \square$) 해시주소를 만듦.

③ 공간 제곱 함수: 탐색 키를 제곱해 그 값의 중간 부분에 있는 숫자로 해시주소 생성.

④ 숫자 분석: 탐색 키의 특징은 알 때 사용. (ex. 학번. 앞은 동일하고 아는 특징이 나뉘는 뒤를 이용함)

⑤ 탐색 키가 문자열: 모든 문자열을 가지고 아스키 코드로 변환해 사용.

이때 곱할 더하면 숫자만 다른 같은 문자열인 경우 구분 함수 없으므로

각 아스키 문자에 상수를 거듭제곱한 식 사용. (ex. $\square \cdot C^N + \square \cdot C^{N-1} + \square \cdot C^{N-2} \dots$)

제곱의 횟수.

* 특징: 가장 기본 특징 = 두 해시값이 다르다면 원래 데이터도 다르다.

(같은 해시값을 갖더라도 원래 데이터가 같음은 보장하지 않는다.)

① * 장의: 양의의 적의 데이터로 고정된 길이의 데이터로 매핑하는 알고리즘

* 응용: 암호학에서 일방향 해시 함수로 무결성 확인 한 예 사용. (메시지 내용에 변경이 없을)

→ 해시 함수로 생성된 해시값 ← 이걸로 메시지 내용 검증됨

→ 메시지 데이터도 알고리즘 (MD) → 이걸 통해 메시지의 무결성을 검증하는 방법, 일방향 해시 함수를 "인출 메시지"

② DB에서 인덱스 만들 때 (해시 인덱스) 사용.

→ 인덱스 생성 시 데이터베이스 관리 시 효율적 검색 / DB 성능 향상에 사용.

→ 해시 테이블 기반 / 인덱스 구성하는 모든 관점 이용 시 유용. → 해시 인덱스 이용함

ex. B-Tree 인덱스 → 키값 이용함

보안의 4대 요소	Authenticity [정확성]	Confidentiality [기밀성]	Integrity [무결성]	Access Control [제한성]	부인방지(불변성) Non-repudiation
가장 중요한 것 → 데이터의 신뢰성	사용자 인증 (ID + Password 사용) (자신 인증 → MAC, 디지털 서명 등 데이터의 신뢰성 확보)	정확한 기밀이 어느 정도 보장될 수 있는가. 원래 데이터가 데이터 전송 후 암호화/복호화 기술이 이루어졌는지, 변경 데이터의 등 여부 확인. (TCI/IT 공개 표준으로 보충됨)	데이터의 무결성. 데이터가 전송 중 변조되지 않았는지 확인.	데이터 시스템에 대한 비인가 접근 차단 및 권한 관리.	부인방지(불변성) Non-repudiation

암호화 알고리즘에서... ← 평문 데이터를 암호화 한 후 암호된 사용자 계를 사용하여 해당 암호문을 복호화해 기밀성 유지.

ex. DES (Data Encryption Standard)
암호화 시키는 Key = 복호화 시키는 Key 같을 때 → 대칭키 암호화
→ 암호에 취약

ex. RSA
→ 비대칭키 암호화
= 공개키 암호화 방식
→ 공개키 암호화 알고리즘 중 하나.

→ 평문을 복호화한 형태로 변환해 기밀성 유지. 암호화 복호화가 분리되어 있음. / 단: 암호 공개가 외부에 알려지면 안됨.

보안에서 무엇을 꼭 지켜야 할지 같이 꼭 조건 먼저 말할 것!

ex: Diffie-Hellman의 암호 교환 방식: 타인에게 알려지지 않은 정보를 두 사람이 교환하는 것만으로 공통의 비밀번호를 만들어내는 방법.

* Radix Tree, Radix Trie 모두 높이 = $\log N + 1$ / complexity: 삽입, 검색, 삭제 모두 = $O(\log N)$

→ 매우 좋은 성능을 나타내는 검색구조 / 비교적 균형이 유지되는 효율적 검색구조 / 비트배열 이용하는 트리구조

→ 기억장소가 충분할 경우 가장 효율적 (기수정렬) (키이진, 기수정렬) = Bucket Sort 같은도 함

11. B-Tree

- Balanced Tree: 이진트리의 문제점(좌우균형 맞지 않으면 비효율적) = degenerated case를 해결한 균형적 트리 / 삽입, 삭제 시 필요하면 스스로 균형을 유지 / 항상 $O(\log N)$ 검색 성능

/ 종류 = AVL Tree, B-Tree, Red-Black Tree 등

- B-Tree: 하나의 노드에 여러 자료가 배치되는 트리구조 / 정렬된 데이터를 보존함

/ 한 노드에 M개의 자료가 배치되면 M차 B-Tree (한 노드에 들어갈 수 있는 최대 자료개수 = M개)

/ 한 노드에 M개의 자료, M+1개의 자식을 가질 수 있음 모든 leaf node는 같은 레벨에 위치하여 균형을 유지 (노드 개수 같은 거임)

- M차 B-Tree 높이 = $\log_{M/2} N$ 이하 / 각 노드는 최소 M/2개의 자료를 가짐

- 검색 시 각 노드 당 최대 M번의 순차검색

- complexity: 삽입, 검색, 삭제 = $O(\log N)$

$$T(N) < c * M * \log_{M/2} N$$

- B-Tree는 외부검색에 효율적인 검색구조 => 하나의 노드 크기를 디스크 입출력 단위(블록)의 크기로 함

/ 순차검색과 트리검색의 이점을 취함 => B-Tree는 Balanced Tree로 Worst case가 없다

/ 쿼리정렬에서 소구간에 대한 삽입정렬이 성능이 좋음

* B+ Tree: 모든 레코드들이 트리의 최하위 레벨에 정렬되어 있고 트리 내부 블록에는 키들만 저장됨

/ 파일시스템 같은 블록기반 스토리지에서 저장데이터의 효율적 검색에 유용함

12. multi-way search tree (m-way search tree)

- 2-원 탐색 트리라고 부르기도 함 / 이진탐색트리(=2-원 탐색 트리라고도 부름)의 일반화된 형태

- 서브트리 개수: 최대 m개 / 각 노드의 키 개수: 최대 m-1개 / 각 노드마다 서브트리 m개 가질 수 있음

- 장점: 분기율을 높임으로써 트리의 높이가 낮아지게 되고 탐색시간이 줄어듦

- 단점: 노드를 삽입, 삭제할 때 트리의 균형을 유지하기 위해 복잡한 연산이 필요함

- m-원 트리: 높이가 낮고 폭이 넓음 / 한 단계 탐색과정 진행할 때 마다 노드의 탐색 수가 약 1/m 로 줄어듦

BUT binary tree: 높이가 높고 폭이 좁음 / 한 단계 탐색과정 진행할 때 마다 노드의 탐색 수가 1/2 로 줄어듦

- 목작: 값을 가로로 많이 배치하여 디스크 블록 읽기를 통해 한번에 읽어들이므로 디스크 액세스 수를 최대한 줄임

방문해야 할 노드 수를 줄이는 것은 결국 디스크 액세스 수를 줄이는 것임 (leaf 노드만 찾아다니는 게 아니라 루트까지 다 방문하기)

- m-원 트리 종류(예가 균형탐색트리 Balanced Search Tree): AVL tree, B-tree, B+ tree 등

→ $O(\log N)$

13. AVL Tree

- Adelson-Velskii and Landis가 제안 / 각 노드의 왼쪽 서브트리와 오른쪽 서브트리의 높이가 서로 균형을 이루는 트리

- 모든 노드의 서브트리 높이 차이가 1 이하인 트리

/ 높이 차이가 2 이상이 되면 노드들 스스로 재배치하여 균형인 상태를 만드는 트리

- 균형인수(balance factor) = 왼쪽 서브트리 높이 - 오른쪽 서브트리 높이 = 1 인 트리

- balance factor가 2이상 인 경우 4가지: LL, RR, LR, RL 방법을 통해 회전시켜 균형을 맞춰줌

- complexity: $O(\log N)$

14. Red-Black Tree

- 이진 탐색 트리 중 하나(좌<root<우)로 가장 효율 좋은 트리 / 노드의 색이 Red or Black인 트리 / root와 leaf node(nil값)는 모두 Black / Red 노드의 자식은 모두 Black / Black 노드의 자식은 Red or Black / root 노드부터 leaf 노드까지의 Black 노드 개수는 어느 경로에서나 동일

- root node부터 가장 먼 경로까지의 거리가 가장 가까운 경로까지의 거리의 두 배 보다 항상 작다

=> 균형이 잡힌 트리 / 모든 worst case에서 complexity가 트리의 높이에 따라 결정되므로 보통의 BST보다 효율적

- rotation은 AVL tree의 기본 operation / 삽입 경우 3가지 있음

1. 삼촌노드 Red : 부모노드, 삼촌노드 -> Black, 할아버지 노드 -> Red, 할아버지를 삽입노드로 생각하고 루트까지 1번 과정 반복

2. 삼촌노드 Black, 삽입노드가 부모의 오른쪽자식 : 부모 기준 좌회전, 부모였던 노드가 삽입한 노드로 생각하고 3번 과정 실행

3. 삼촌노드 Black, 삽입노드가 부모의 왼쪽 자식 : 부모노드 -> Black, 할아버지 노드 -> Red, 할아버지 노드 우회전

- complexity: $O(\log N)$

MST 만드는 방법 ← 무작정 알고리즘 → 모든 신장트리를 고려, 그 중 최소비용이 되는 것을 선택 $\Rightarrow$ 매우 비효율적 $O(n^{n-2})$
탐욕 알고리즘 (Greedy Algorithm) $\leftarrow$ =

- ↓
- 최종해답을 얻지 못하는 동안 다음의 3단계 반복작업.
 - 선정 절차: 적절한 후보해 선정 절차에 따라 하나의 이음선 선정.
 - 적정성 점검: 선정된 이음선을 E에 추가해도 사이클이 생기지 않으면 E에 추가.
 - 해당 점검: 안 들어본 트리가 신장트리이면 문제 해결.
- ↓

$\therefore$ 다른 방법.

탐욕 알고리즘 사용한 대표작인 알고리즘 = Prim's Algorithm.

15. Graph

- 정점(vertex=node)과 간선(edge)로 이루어진 자료구조 / 객체간의 연결을 모델링하는 자료구조
- 경로(path) : 한 노드에서 다른 노드까지의 길 / 순환(cycle) : 처음 정점과 끝 정점이 같은 경로
- 차수(degree) : 하나의 정점에 달아있는 간선의 수 => 진입차수(in-degree)/진출차수(out-degree) (방향그래프만)
- 방향(directed)그래프: 출발노드와 도착노드가 정해짐 / 무방향(undirected)그래프: 출발노드와 도착노드의 구분없음
- n개 꼭짓점 가진 무방향 그래프의 최대 변의 수 = $n(n-1)/2$ = 완전그래프(complete graph)
- 인접하다(adjacent) = 두 꼭짓점(정점)이 변으로 이어져 있는 상태
- Spanning Tree : 신장트리 = 그래프의 부분 그래프 중 모든 정점과 연결된 부분 그래프(cycle이 없는 그래프)
- Minimum Spanning Tree(최소신장트리) : 신장트리에서 가중치(간선 비용의 합) 합이 최소인 트리
/ (ex. 지하철역 최단거리, 최소비용으로 통신망연결 등) / 이걸 위해 Dijkstra 알고리즘 존재(시작점 제외한 모든 정점 시작점 가리킨다 가정 / 이어지지 않은 정점은 무한대 표시(무한대 비용)하고 가중치 작은 정점을 찾아 잇는 것)
- 그래프 순회/탐색 방법 : DFS(깊이 우선 탐색) / BFS(너비 우선 탐색)
/ (현재 위치를 스택에 넣어 기억하고, 방문하지 않은 인접리스트에 대해 계속 처리함)
- Depth First Search : 시작점을 기준으로 가장 아래까지 이동하면서 탐색하는 방식 (↑↓ 인접점탐색)
/ 인접한 꼭지점을 순서대로 방문하는 방식
- Breadth First Search : 인접한 꼭지점을 순서대로 방문하는 방식
/ 인접한 꼭지점 순서대로 방문하기 때문에 큐를 만들어서 넣는 순서대로 방문함 (→ 순차진행)
- 그래프 표현방법 : 인접리스트(Adjacency List) / 인접행렬(Adjacency Matrix)
- 인접리스트 : 한 정점에 간선으로 연결된 다른 정점들을 연결리스트로 연결해 놓는 방법
/ 필요한 만큼만 메모리를 사용하므로 희소그래프(sparse: 정점에 비해 적은 간선수)의 표현에 유리함
/ 적은 메모리 공간을 요구하나 행렬 구조에 비해서 느림
- 인접행렬 : 정점의 수가 V인 그래프를 $V \times V$ 행렬을 이용하여 표현
/ $V \times V$ 만큼의 메모리를 사용하므로 밀집그래프(그래프가 조밀하게 구성)의 표현에 적합함
/ 많은 메모리 공간을 요구하나 리스트 구조에 비해서 빠른 접근 제공(random access 가능한 것)

	Adjacency list	Adjacency Matrix
저장공간	$O(V+E)$	$O(V^2)$
Vertex 추가	$O(1)$	$O(V^2)$
Edge 추가	$O(1)$	$O(1)$
Vertex 삭제	$O(E)$	$O(V^2)$
Edge 삭제	$O(E)$	$O(1)$

노드의 개수를 V라고 하고 링크의 개수를 E라고 해보자. 이 때, 인접 행렬은 $O(V^2+E)$ 의 시간을 가지고, 인접 리스트는 $O((V+E) \cdot \log V)$ 의 시간을 가진다. 밀집 행렬일 경우에는 V에 비해 E가 커지기 때문에 인접 행렬의 $O(V^2+E)$ 가 더 유리하고, 반면 희소 행렬일 경우에는 E에 비해 V가 크기 때문에 인접 리스트의 $O((V+E) \cdot \log V)$ 가 더 유리하다. ✓ 설명이랑 위의 표랑 참고하기

MST 표현 알고리즘

① Kruskal

- 최소비용을 가진 edge들을 차례로 탐색해 나가면서 cycle이 없는 경로를 찾는 방법.
- $O(E \cdot \log V)$ (= $O(E \log E)$)
(= 트리 정렬하는데 걸리는 시간 $O(E \log E)$ + cycle생성 여부 검사 시간 $O(E) = O(E \log E) = O(E \log V)$) dense tree: $O(N^2 \log N)$

② Prim

- 임의의 정점을 기준으로 비용이 가장 적은 edge를 탐색해 나가는 방법, 역시 cycle이 없는 경로를 찾음.
- 밀집그래프(dense graph)에서 Optimal하다. ($|E| \approx |V|^2$ 인 경우 Kruskal 보다 빠름) dense tree: $O(N^2)$

이진 힙(Binary Heap)이름한 경우 = $O(E \cdot \log V)$

① 인접리스트 = $O(E \log V)$

② 인접행렬, (원소) = $O(V^2)$

③ 피보나치 힙 ① 인접리스트 = $O(E + V \log V)$

④ Sollin 알고리즘: 최소한의 비용을 갖는 subset을 조합하는 방법 (= forset)
→ $O(N^2 \log N)$ → 병렬 알고리즘에서 가장 높은 효율을 가짐.
PPL 병렬 컴퓨터(여러개의 프로세서로 사용하여 여러개의 명령을 동시에 실행)
→ 여기서 실행되도록 고안된 알고리즘.
Parallel Pattern Library

운영체제

1. Thread

- Thread : 프로세스 내에서의 작업 단위 / 시스템의 여러 자원을 할당받아 실행하는 프로그램의 단위
- 장점: 시스템 자원을 공유해서 기억장소 낭비 줄임 / 프로그램 응답 시간 단축됨 / 자원 공유가 용이함

② 하나의 프로세스를 여러개의 스레드로 생성 → 병행성 증진 (parallel) (concurrency)
③ multi-process system일 때, 각 스레드는 서로다른 프로세서를 병렬적으로 실행, 동시성 향상

④ 단점: 병행성 문제 → 두 개 이상의 스레드가 어떤 객체에 있는 하나의 데이터에 접근할 때,

/ 해결법 ⇒ lock = 상황에 따라 한 스레드가 사용중인 자원을 특정작업이 끝날 때까지 다른 스레드에서 쓸 수 없게 만들
synchronized = 어떤 메소드를 한 번에 한 스레드만 접근가능 하도록 하는 것

- Process : 일반적으로 실행중인 프로그램이라고 함

PCB((Process Control Block): 프로세스에 대한 중요 정보 저장)를 가짐(프로세스 생성시마다 PCB생성)

Context-switch(문맥교환) : 프로세스가 다음에 실행할 프로세스로 전환되는 과정. 이전 프로세스에 대한 정보적재
스레드와 프로세스 비교

- 공통점 : 모두 여러 흐름이 동시에 진행된다는 것

- 차이점 : 프로세스 = 독립적으로 실행되며 각각 별개의 메모리를 차지하고 있음

스레드 = 프로세스 내 메모리를 공유해 사용 = 경제적

/ 문맥교환의 속도가 빠르다(프로세스에 비해 보존해 두어야 할 정보가 상대적으로 적어 시간이 짧음)

2. 가상 메모리 운영방법

1. 페이징 기법

- 프로그램을 동일한 크기로 나눈 단위를 페이지라 하며 이 페이지를 블록으로 사용하는 기법
⇒ 동일한 크기로 나뉜 프로그램을 주기억장치의 영역(페이지 프레임)에 적재시켜 실행 / 내부 단편화 발생

2. 세그멘테이션 기법

- 프로그램을 가변적인 크기로 나눈 단위를 세그먼트라 하며 이 세그먼트를 블록으로 사용하는 기법

⇒ 기억공간 절약하기 위해 사용 / 외부 단편화 발생

- 내부 단편화 : 분할된 영역 크기 > 할당될 프로그램 크기 → 사용되지 않고 남은 빈 공간

- 외부 단편화 : 분할된 영역 크기 < 할당될 프로그램 크기 → 사용되지 못하고 남은 빈 공간

3. Paging

- 페이지 크기: 페이징 기법 사용시 프로그램을 페이지 단위로 나눌 때 시스템에 미치는 영향

小: 페이지 단편화 ↓ (감소) / 매핑속도 ↓ (테이블 크기 ↑) / 입출력 효율 ↓ (자주 접근, 시간 ↑) / 적재이동시간 ↓
/ 효율적인 working set (locality 일치-가까운 시간 내(temporal), 근처 페이지(spatial) 계속 참조), 장치의 효율 ↑

- 大: 페이지 단편화 ↑ / 매핑속도 ↑ (테이블 크기 ↓) / 입출력 효율 ↑ (디스크 접근수 ↓, 시간 ↓) / 적재이동시간 ↑

/ 불필요한 내용까지 적재될 수 있음

- Locality(국부성) : 프로세스가 실행되는 동안 주기억장치를 참조할 때 일부 페이지만 집중적으로 참조하는 성질

- Working Set : 프로세스가 일정 시간동안 자주 참조하는 페이지들의 집합 / 주기억장치에 상주시켜서 효율 ↑

- Page Fault(페이지 부재) : 프로세스 실행시 참조할 페이지가 주기억장치에 없는 현상

/ 페이지 부재율에 따라 주기억장치에 있는 페이지 프레임 수를 조절해 적정 수준으로 유지

- 페이지 교체 알고리즘 : 페이지 부재 발생 시 어떤 페이지 프레임을 선택하여 교체할 것인가 결정

- OPT(OPTimal replacement: 최적 교체) : 앞으로 가장 오랫동안 사용하지 않을 페이지 교체 / 미리예측=가능성희박

- FIFO(First In First Out) : 적재된 시간 기억해 먼저 들어온 순서대로 페이지 교체

/ 페이지 프레임수 ↑ 시켰지만 페이지 부재 ↑ 더 많이 발생 = 모순 발생

- LRU(Least Recently Used) : 최근에 가장 오랫동안 사용하지 않은 페이지를 교체 / 오버헤드 발생 / 구현 어려움

- LFU(Least Frequently Used) : 사용 빈도가 가장 적은 페이지를 교체 / 사용되지 않은 페이지가 프레임 계속 차지

- Thrashing(스래싱) : 프로세스의 처리 시간 < 페이지 교체에 소요되는 시간 ⇒ 전체 시스템 성능 저하됨

⇒ 방지법: working set 유지, 페이지 부재 빈도 조절, 다중프로그래밍 정도 적정수준 유지 등

4. 요구 페이징(Demand Paging) 07

- 필요한 프로그램만 메모리에 적재하는 방법 / 가상 메모리 시스템에서 많이 사용
- 일반적으로 Swapping 기법(하나의 프로그램 전체를 주기억장치에 할당해 필요시 다른 프로그램과 교체하는 것)과 유사
- 미적용 : 사용하지 않을 메모리도 페이지에 적재시켜 메모리 낭비
- 적용 : 사용되지 않을 메모리 페이지를 읽어 들이는 것 예방 => 교체시간↓, 기억공간↓, 다중프로그래밍 정도↑

5. 프로세스 스케줄링 07

- 스케줄링 목적: CPU나 자원을 효율적으로 사용하기 위해(응답시간 최소화, 처리량 최대화)
- 비선점(Non-preemptive) 스케줄링 => 일괄처리시스템
 - : 이미 할당된 CPU를 다른 프로세스가 강제로 빼앗아 사용할 수 없는 스케줄링 기법 (리눅스: FCFS)
 - FCFS(First Come First Service) : 큐에 들어온 순서대로 처리 / 공정성 유지 but 효율 떨어짐(중요도, 작업길이)
 - SJF(Shortest Job First) : 긴 프로세스가 실행시간 짧은 프로세스에게 할당순위 밀려서 무기한 연기상태 발생
 - 우선순위(Priority) : 우선순위가 가장 높은 프로세스에 CPU 먼저 할당 / 낮은 우선순위 프로세스 무한연기, 기아상태 => Aging 기법: 일정시간 지나고 우선순위를 한단계씩 높여 가까운 시간내에 자원할당 받게해 무한연기/기아상태 예방
- 선점(Preemptive) 스케줄링 => 대화식시분할시스템
 - : 하나의 프로세스가 CPU를 할당 받아 실행하고 있을 때 우선순위가 높은 다른 프로세스가 CPU를 강제로 빼앗아 사용가능 => 장점: 우선순위 높은 프로세스 빠른 처리 가능 / 단점: 많은 오버헤드 초래
 - RR(Round Robin) : 타임 슬라이스(or 쿼텀)라는 단위로 공평하게 프로세스 별로 할당량 부여하여 처리하는 방식 / 시분할 시스템에 적합함 / 타임 슬라이스 大: FCFS, 小: 문맥교환↑, 오버헤드↑, 속도↓

6. 디스크 스케줄링 07

- : 사용할 데이터가 디스크 상의 여러 곳에 저장되어 있을 때 데이터를 액세스하기 위해 디스크헤드의 경로를 설정하는 기법
- 목적: 탐색시간의 최적화 / 처리량 최대화 / 응답 시간 최소화
- FCFS = 데이터 서비스공평성 보장 / but 헤드 이동 거리가 길어질 수 있음
- SSTF(Shortest Seek Time First) = 일괄처리시스템에 유용 / but 응답시간 편차가 커서 대화형시스템에 부적합
- SCAN = 오버헤드 적을 때 가장 효율적 / and SSTF의 탐색 시간 편차 해소

7. 병행 프로세스 07

- 두 개 이상의 프로세스가 동시에 존재하며 실행 상태에 있는 것(다중 프로그래밍 기법 이용)
- 문제점: 한정된 자원(CPU, 메모리, 디스크, I/O장치)에 대한 사용 순서 문제 발생(ex. 공유프린트출력물 순서 섞임)
=> 해결법: 임계구역 / 상호배제 / 동기화기법
- 임계구역(Critical Section): 한 번에 한 프로세스만 자원 or 데이터를 사용하도록 지정한 영역
 - 특정 프로세스가 독점 불가 / 임계구역 진입요청 들어오면 일정 시간 내에 수락해야함
- 상호배제(Mutual Exclusion): 한 프로세스가 메모리 사용하고 있을 때 다른 프로세스가 사용 못하게 배제시키는 것
 - 교착상태(Dead Lock) = 상호배제 시 나타나는 문제점 => 해결법 : 예방 / 회피
- : 두 프로세스가 이미 자원을 점유하고 있는 상태에서 서로의 자원을 요구하여 무한정 기다리게 되는 현상
- 동기화기법: 프로세스 처리 순서를 결정 => 세마포어(각 프로세스에 제어 신호 전달) 공유 자원에 대한 접근 권한 시. P, V 원리
/- 모니터(임계구역과 유사 - 자료 추상화, 정보은폐 개념)
동시 실행을 원하는 프로세스만 이용 가능
상호 배제
가능 시점으로 넘어가기 위해 이용 가능
wake up
진입
준비

8. 분산처리 시스템 07

- 여러 대의 컴퓨터로 작업을 나누어 처리하는 방식 / 그 내용이나 결과를 통신망을 이용해 상호 교환
- 공유도↑, 연산속도↑, 신뢰도↑, 보안성↓, 개발 복잡, 개발 비용↑

네트워크

1. OSI(Open System Interconnection) 7계층

: 통신이 일어나는 과정을 7단계로 구분한 것 (개발의 참조모델, 프로토콜의 표준적 뼈대 제공)

- 1 Physical layer(물리 계층)
 - = 두 컴퓨터 간 물리적 연결을(정의)하는 계층 / 데이터를 물리매체 상으로 전송 / 장비들 간 Bit 전송
- 2 Data link layer(데이터링크 계층)
 - = 물리적 계층을 통한 데이터 전송에 신뢰성 제공 / 물리적 주소 지정(MAC address) / Frame 안전하게 전송
- 3 Network layer(네트워크 계층)
 - = 경로 선택 및 라우팅, 논리적 주소(IP) 정의하는 계층 / 패킷 이동경로 결정 / Packet으로 분할하여 전송
- 4 Transport layer(전송 계층)
 - = 상/하위 계층간 인터페이스 기능 및 메시지 전달 / 흐름제어 및 오류제어 / TCP, UDP 프로토콜 / Segment
- 5 Session layer(세션 계층)
 - = 통신하는 두 호스트 간 최초연결 및 연결유지 담당 / 세션을 열고 닫고 관리 / dialogue관리(= 대화 = 통신)
- 6 Presentation layer(표현 계층)
 - = 전송 데이터 포맷 결정 / 데이터 포맷 상호 변환, 압축(compression) 및 암호화(encryption)수행 / ex. jpeg, avi
- 7 Application layer(응용 계층)
 - = 사용자가 이용하는 네트워크 응용프로그램 / 응용프로세스 상호간 정보교환 창구/telnet(23), http(80), ftp(21) protocol

2. TCP/IP 모델

: OSI7계층=장비 개발자가 어떻게 표준을 잡을지 결정 / TCP/IP=실질적으로 사용되는 프로토콜에 대한 것(신뢰도↑)

- 1, 2 = Network Access layer(네트워크 접속 계층) / - 3 = Internet layer(인터넷 계층)
- 4 = Transport layer(전송계층) ICMP, 흐름/오류제어 / - 5, 6, 7 = Application layer(응용 계층)

3. 가상주소 물리주소 차이

- MAC address: 각 장비들이 정확한 통신을 위해 네트워크상에서 서로 구분하는 역할을 해주는 것 / 각각의 네트워크 장비의 인터페이스는 고유의 MAC 주소를 가짐

← TCP/IP 프로토콜 사용하는 네트워크에서 IP 주소 사용하여 통신. but, 최종적으로 MAC 주소 사용하여 데이터 전달

- ARP(Address Resolution Protocol) : IP → MAC ↔ RARP(reverse) : MAC → IP (인터넷계층3)
- 논리적인 가상주소를 물리적인 실기억주소로 변환하는 것 = 주소 사상 = 매핑(Mapping)

4. TCP 역할 / IP 역할

- TCP(Transmission Control Protocol) 신뢰성 ↑
 - : 연결 지향성 프로토콜 / 신뢰적 / 장: 보장된 세그먼트 전달 / 단: 연결을 위한 초기 설정 시간 소요
- UDP(User Datagram Protocol) 신뢰성 ↓
 - : 비연결형 프로토콜 / 비신뢰적(데이터 도착여부 모름) / 빠름 / 신뢰성보다는 고속성 요구되는 응용에 사용
- IP(Internet Protocol) 3
 - : 비연결형 프로토콜 / 데이터 전송 / 발신지(송신지)부터 목적지(수신지)까지의 라우팅 경로를 결정하는데 사용

5. 데이터 체인 방식

- 인터럽트 발생 시 우선순위 판별방법: S/W=폴링(Polling)기법 / H/W=데이터 체인(daisy-chain)기법

S/W 폴링: 인터럽트 요청한대로 차례대로 검사하는 방식 / 응답속도 느리다

H/W 데이터 체인: 인터럽트 우선순위에 따라 직렬로 연결하는 방식 / 응답속도 빠르다

= 연결방식이 데이터체인!

S/W	H/W
지연	고속
간단	복잡
정리하기	바깥에서

polling: 프로그램이 이연상태에서 지속적으로

daisy-chain: 연속적으로 판별되어있는 하드웨어가

*바인딩 (Binding): 함수 호출과 실제 함수를 연결하는 방법 // 프로그램 실행 시 각 함수 호출은 특정 함수를 호출하는 리징.
기본적으로는 정적 바인딩 실행.

종류

정적 바인딩 (Static Binding) [컴파일 타임에 호출될 함수를 결정. (컴파일 시)
객체나 타입을 보고 호출할 함수 결정.

동적 바인딩 (Dynamic Binding) [런타임(실행 중)에 호출될 함수를 결정. (프로그램 실행 시)
virtual 메서드를 사용해 가상 함수를 정의해 주함.
호출 함수의 결정은 객체의 가상 함수 table 사용.

*가상 함수 (Virtual function): virtual 메서드를 붙여 정의된 함수는 런타임 시 포인터가 가리키는 타입에 따라 호출할 함수 결정.
포인터가 어떤 객체를 가리키는가에 따라 호출될 함수 결정.
이를 통해 다형성 구현.

이산수학

* X에서 Y로의 함수인 $f: X \rightarrow Y$

- 정의역(Domain): 함수의 값이 정의된 집합 / x값 전체의 집합
- 공역(Codomain): 함수의 값들이 속하는 집합 / y값 전체의 집합
- 치역(Range): 정의역에 대응하는 공역의 원소를 모아둔 집합 / 함수의 모든 출력 값의 집합 / 치역 $\subset$ 공역

1. 역함수

- 함수 $f: X$ 가 Y 에 대해 일대일대응일 때, 함수 Y 의 각 원소 y 에 대해 $f(x) = y$ 인 원소 x 는 단 하나 존재합니다. 이 원소 x 에 대응하고 Y 를 정의역, X 를 공역으로 갖는 함수를 f 의 역함수라 하고 $x = f^{-1}(y)$ 로 나타낼 수 있습니다.
- *조건=> 일대일대응 / 증가함수(x 값 $\uparrow \rightarrow$ 함수 $f(x)$ $\uparrow$) or 감소함수(x 값 $\uparrow \rightarrow$ 함수 $f(x)$ $\downarrow$)

일대일 대응 $\rightarrow f(x)=y$ 인 x 는 유일 $\rightarrow x$ 에 대응하고, $\forall y \in Y$, x 공역인 함수로 역함수 가능
 $\Rightarrow x = f^{-1}(y)$

2. 멱집합(Power Set)

- 어떤 집합 A 에 대하여 A 의 모든 부분집합들로 이루어진 집합을 A 의 멱집합이라고 합니다. A 의 원소의 개수가 n 개라면 멱집합의 개수는 2^n 개입니다. $P(A) = 2^A$ 로 나타낼 수 있습니다.
- *증명=> A 의 어떤 부분집합을 만들 때 각 원소에 대해 그 원소가 들어가는 경우와 그렇지 않은 경우 2가지로 나눈다. 이렇게 원소마다 부분집합에 대한 2가지 경우의 수가 생기고 각 사건이 독립적이므로 각 원소 수만큼 곱한다.
- cf. 멱함수: ' $y=ax^n$ '과 같은 식으로 나타낼 수 있는 함수.

3. 수학적 귀납법

- 자연수 n 에 대한 명제 함수 $p(n)$ 에 대해 $p(1)$ 참이고 임의의 자연수 k 에 대해 $p(k)$ 가 참이라 가정합니다. 이 때 $k+1$ 에 대하여 $p(k+1)$ 이 참 일 때 모든 자연수 n 은 함수 $p(n)$ 에 대해 참임을 증명하는 방법입니다.
- *조건=> 자연수
- cf. 귀류법(proof by contradiction) = 모든 경우가 성립하지 않을 때 자주 사용하는 증명법
(p 가 거짓이면 q 가 거짓이다. 가정이 잘못되었으므로 $\therefore p$ 가 참이어야함)

4. 릴레이션

- 두 집합의 원소들 사이에 어떤 관계를 이루는 부분집합을 릴레이션이라고 합니다.
(유사한 대상들을 관련시키는 성질들의 특정 조합을 릴레이션이라고 합니다.)
- *표현법=> 카르테시안 곱($A \times B$)의 부분집합 형태 / 화살표 선도 / 표 / 방향성 그래프 등으로 표현 가능
(directed graph=>인접행렬/리스트)
- 이런 표현 형태들을 '집합 A 에서 집합 B 로의 관계'라고 말합니다.

5. 동치관계(equivalence relation)

- 어떤 집합 A 에 대한 이진관계가 반사적, 대칭적, 전이적이면 집합 A 에 대한 동치 관계라고 합니다.
- 이진관계(집합 A 의 순서쌍 집합)
- 반사적(모든 x 가 자기 자신과 관계됨) / 대칭적(x 와 y 가 관계되면 y 는 x 와 관계됨)
- 전이적(x 가 y 와 관계되고 y 와 z 와 관계되면 x 는 z 와 관계됨) / 비대칭적(x 와 y 가 같을 때만 대칭 관계가 성립)

6. 집합의 상등(Equality of Sets)

- 서로 같은 두 집합을 말합니다. 두 집합 A, B 가 있을 때 A 의 원소가 모두 B 에 속하고, B 의 원소도 모두 A 에 속하면 집합 A 와 집합 B 는 서로 같다고 하고 이를 집합의 상등이라 말합니다.

수차 접근 ① ② 만지 해킹권민국하는 적외선 감고출 제리만수행

*파일시스템: 파일을 생성하여 자료들은 추가, 수정, 삭제, 검색 등의 조작할수있는 SIW시스템

↓
독립된 파일 단위로 (업무와 관련된) 데이터를 저장하므로 데이터 중복성과 데이터 종속성 발생
②
독립성

*데이터베이스시스템: 파일시스템의 문제점을 해결하며 데이터의 독립성, 데이터의 일관성 및 무결성 유지

↓
특징: 실시간 접근 / 지속적인 변화 / 공유성 / 내용에 의한 참조
(연계데이터 공유데이터) (신뢰, 속내 명단) (여러사람들) (주요사/주요분야나 값에 따라참조)

DBMS: 응용프로그램과 데이터베이스 사이의 중재자.

Random Access 정의, 조작, 제어 기능을 함.

(사용자 R 제어해 임의데이터 읽기) → ④ 안정성과 편리성 제공
데이터의 추가, 수정, 삭제 등이 용이함

장점: 독립성 보장, 무결성 보장, 일관성 유지, 데이터통합관리, 데이터 실시간제

단점: 비용↑, 자료처리 복잡, 시스템 복잡 (병행, 분산), 전문인력 부족

*스키마 = 데이터베이스의 구조와 제약조건에 대한 명세를 기술한 것.
(=메타데이터)
데이터사전 (=시스템카탈로그)에 저장됨

외부스키마: 사용자 관련, 여러개 존재
- 개념스키마: 인본적스키마, 전체 뷰, DB전체논리구조, 무결성 규칙
내부스키마: 실제데이터 저장, DB 물리적구조

① 캡슐화(Encapsulation): 데이터와 함수 등 객체와 관련된 것들을 하나로 묶는 것을 말한다.

1~4 객체지향 언어의 특징? 흔히 정보은닉과 함께 연관지어 사용되는 개념으로 외부에서 알 필요가 없는 데이터나 행위는 보이지 않도록, 실행흐름이 외부에 드러나지 않도록 한다. / 재사용성 향상

② 추상화(Abstraction): 인터페이스와 구현을 분리하는 것 / 구현을 통해 객체가 가진 특성 중 필수속성만으로 객체를 묘사하고 유사성만을 표현하여 세부적 사항은 각 객체마다 다르게 구현하도록 할수있다.
(C++에서는 가상함수를 통해 (상속을 이용) 추상화 가능)

③ 상속(Inheritance): 클래스기능을 확장하는 (예)생병증 하나 / 기존 클래스를 수정하지 않으면서도 이미 정의되어 있는 내용을 확장해서 사용할수있는 방법을 제공한다. / 코드의 간결성↑ (구현시 비슷한 속성의 공통 구현방지) / 부모클래스 → 자식클래스 관계

④ 다형성(Polymorphism): 서로 다른 타입에 대해 동일한 방식으로 접근할 수 있도록 하나의 인터페이스를 제공하는 것이다.
하나의 형태로 여러개의 내용을 가짐 (즉, 동일한 타입의 객체의 동일한 메소드가 다른일을 하는 것)
재사용성과 유연성 확장 (인터페이스를 통한 유연한 기능 확장 및 클래스 추가 가능) / 동일한 메세지에 대해 서로 다르게 행동할
C언어에서.. ex. int + double처럼 서로 다른 타입이나, + 연산 자체는 모든 기본타입에 대해 동일하게 자원하고있음
C++에서 더 다양한 다형성 제공 ex. 가상함수, 템플릿, 오버로딩 등... 다. 메세지 특정 선택하는데, 리시버가 → 부호제기 등 5월 것

*오버로딩(overloading): 중복정의의라고도 할수 있다. 클래스 안에 같은 이름을 가진 함수를 여러개 정의할 수 있다.
단, 매개변수의 타입, 개수가 달라야 함.

*오버라이딩(overriding): 상속관계에 있는 클래스들 간 같은 이름의 함수를 정의 하는 것. 함수의 재정의라고도 한다.
자식클래스에서 부모클래스의 함수 재정의함. / + 동적바인딩 방식

데이터베이스

- 외부스키마: 사용자가 보는 관점 / 개념스키마: DB전체 논리구조, 제약조건, 무결성 / 내부스키마: DB물리적구조
- DDL: DB정의 및 변경 / DML: 사용자 요구에 따라 검색, 갱신, 삽입 등 지원 / DCL: 정확성, 안정성 유지
- 관계대수: 절차적언어, How, select, project, join, division / 실행할 연산을 명시
- / 관계해석: 비절차적언어, What, 관계대수로 표현가능 / 원하는 결과의 집합론적인 서술을 제시

- DB설계: 요구조건분석 -> 개념적설계(ERD) -> 논리적설계(RDB관계형데이터베이스/정규화) -> 물리적설계 -> DB구축
- E-R(Entity-Relationship) 모델: 개체와 개체 간의 관계를 도식화 한 것 (1:1 / 1:N / N:M)

- 관계형 데이터베이스: 도메인(속성이 가질 수 있는 값들의 집합) / 차수(속성 수) / 카디널리티(튜플 수)

- 슈퍼키: 속성들의 집합으로 구성된 키 / 유일성 O / 최소성 X
- 후보키: 개체들을 고유하게 식별할 수 있는 속성 / 유일성 O / 최소성 O
- 기본키: 주키 / 후보키 중 선택한 키 / 중복 X / NULL X
- 외래키: 다른 릴레이션의 기본키 참조하는 키

- 제약조건(무결성, Integrity): 개체(entity) 무결성: 기본키를 구성하는 키 값은 NULL X, 중복 X

- 참조 무결성: 참조할 수 없는 외래키를 가질 수 없음
- 도메인 무결성: 각 속성 값은 반드시 정의된 도메인에 속한 값이어야 함

- 정규화(Normalization): 종속성, 중복성의 배제, 릴레이션의 분해(이상현상 줄임)가 목적 / 단점: 연산 시간의 증가
- 1NF(도메인이 원자값) / 2NF(부분적합수종속성제거) / 3NF(이행적합수종속성제거) / BCNF(결정자O, 후보키X제외) / 4NF(다치종속성제거) / 5NF(조인종속성이용)

1NF->5NF 정규화 과정

- JOIN 연산: 공통 속성을 중심으로 두 개의 릴레이션을 하나로 합쳐서 새로운 릴레이션을 만드는 연산

=> DB에 부담을 주므로 가급적 사용하지 않는 게 좋음(외래키로 데이터 검색하므로 성능 저하)

- 병행 제어: 목적: DB 공유 최대화 / 일관성 유지 / 시스템 활용도 최대화 / 사용자 응답시간 최소화

↳ 기법 = Locking(로킹): 하나의 트랜잭션이 데이터 액세스하는 동안 다른 트랜잭션은 액세스 불가

로킹 단위 대: 로크 수 소 -> 관리 수월 / 병행성 수준 ↑ / 공유도 ↓
소: 로크 수 대 -> 관리 복잡 / 병행성 수준 ↓ / 오버헤드 ↑

기타

* 함수에서 파라미터 패싱 방법

- call by value (값에 의한 호출)

: 어떤 함수를 호출할 때 전달인자로 그 데이터 값 자체를 전달하는 호출 방법이다.

=> 인자를 넘겨줄 때 마다 메모리 공간을 할당하기 때문에 공간을 소모하나, 원본 데이터에 손상을 주지 않는다.

- call by reference (참조에 의한 호출)

: 어떤 함수를 호출할 때 전달인자로 어떤 특정한 데이터 값이 아닌 그 값에 대한 정보를 전달하는 호출방법이다.

=> 인자를 넘겨줄 때 주소 값을 참조하므로 메모리 공간을 따로 할당하지 않지만, 원본 데이터에 손실을 준다.

* Big O 표기법

- 시간 복잡도 함수에서 불필요한 정보를 제거하여 알고리즘 분석을 쉽게 할 목적으로 시간 복잡도를 표시하는 방법을 빅오표기법(big-oh notation)이라 한다.

(f와 g를 각각 양의 정수를 갖는 함수라 할 때, 만일 어떤 두 양의 상수 a와 b가 존재하고,

모든 $n \geq b$ 에 대하여, $f(n) \leq a \cdot g(n)$ 이면 $f(n) = O(g(n))$ 이다.)

* UNIX에서 프로세스 생성 함수

- fork() => 부모프로세스를 복제하는 함수 => 부모 -> 자식 (부모) -> 자식

=> 결국 트리형태가 된다. / 각각의 프로세스 마다 고유의 pid(process id)를 부여한다.

fork() == 0

- ① 순차상환수로 선언한 클래스는 인스턴스 생성 불가. (= 추상클래스는 인스턴스 생성 불가.)
추상클래스를 상속한 자식클래스는 순차상환수를 구현해야 인스턴스 생성 가능.

- ② STL (Standard Template Language)
템플릿으로 함수 작성 후 호출시엔 호출부의 양의타입을 넣어 그 타입에 맞는 함수를 (컴파일러가 찾아내서) 자동으로 작성한다.